Counterfeit Pharmaceutical Detection

Complete Study Guide

Comprehensive reference for viva, exam, and project demo

Author: Chethan N

GitHub: ChethanNGowda/Counterfei

Technologies: Python 3.12 | PyTorch | EfficientNet-B0 | FastAPI | Ollama | qwen3-vl:4b | qwen3:8b

. Table of Contents

1. Project Overview & Problem Statement
2. System Architecture & Pipeline
3. Primary ML Model: EfficientNet-B0
4. Supporting Models: MobileNetV3, Baseline CNN, GoldFormer
5. Dataset: Collection, Cleaning & Splits
6. Training Process & Hyperparameters
7. Model Evaluation Results
8. Secondary AI: qwen3-vl:4b (Vision Verifier)
9. Secondary AI: qwen3:8b (Evidence Reviewer)
10. Decision Engine (4-Rule Conservative Policy)
11. FastAPI Backend & API Endpoints
12. Image Quality Preprocessing
13. Testing & Automated Test Suite
14. Project File Structure
15. Live Demo: Running the System
16. Limitations & Safety Boundary
17. Future Improvements
18. Potential Exam Questions & Answers

1. Project Overview & Problem Statement

What is the Problem?

Counterfeit pharmaceuticals are a massive global public health crisis. The World Health Organization (WHO) estimates that approximately 10% of medicines in low and middle income countries are substandard or falsified. Counterfeit medicines contain wrong active ingredients, under-dosing, dangerous substitutes, or completely inert fillers. This causes treatment failures, antimicrobial resistance, severe injury, and death.

Detecting counterfeits in the field (pharmacies, supply chains, households) is very difficult because:

- Counterfeits are designed to look identical to genuine medicines.
- Laboratory chemical testing (HPLC, spectroscopy) is expensive and time-consuming.
- Pharmacists and patients in remote areas have no testing equipment.
- Existing tools require expensive handheld spectrometers.

The Project Solution

This project builds an AI-assisted screening system that uses a smartphone camera image of a medicine tablet to provide an INITIAL preliminary screening verdict. It uses:

- A fine-tuned Deep Convolutional Neural Network (EfficientNet-B0) to visually classify tablet images.
- A secondary AI Vision Model (qwen3-vl:4b) for direct visual inspection of the image.
- A secondary AI Text Reviewer (qwen3:8b) to reason over statistical evidence from the classifier.
- A conservative Decision Engine that prevents false certainty and always recommends lab testing.

IMPORTANT SAFETY NOTE: This system performs AI-assisted preliminary screening ONLY. It CANNOT replace laboratory chemical testing. High model confidence does not mean chemical purity is guaranteed.

Output Classification

The system produces one of 3 final screening outcomes:

- LIKELY AUTHENTIC - Both ML and AI agree the tablet visually conforms to genuine characteristics.
- LIKELY COUNTERFEIT - Both ML and AI agree the tablet shows counterfeit characteristics. Sample quarantined.
- NEEDS MANUAL REVIEW / SUSPICIOUS - Low confidence, model disagreement, or poor image quality. Physical lab inspection mandatory.

2. System Architecture & Pipeline

The system is a multi-stage dual-model pipeline. Every uploaded image goes through this exact sequence:

```
INPUT IMAGE (Smartphone Photo of Tablet)
         |
         v
[STAGE 1] OpenCV Image Quality Precheck
  - Minimum Resolution:  >= 64 x 64 pixels
  - Blur (Laplacian Variance): >= 50.0
  - Brightness Mean: 15 to 245 (avoids black/white-out)
  FAIL -> NEEDS MANUAL REVIEW (poor image quality)
  PASS -> Continue
         |
         v
[STAGE 2] Image Preprocessor
  - Resize to 224 x 224 pixels (RGB)
  - Normalize with ImageNet mean=[0.485,0.456,0.406],
                              std=[0.229,0.224,0.225]
         |
         v
[STAGE 3] EfficientNet-B0 (Primary ML Classifier)
  - Produces softmax probabilities: P(authentic), P(counterfeit)
  - Confidence = max(P(authentic), P(counterfeit))
         |
    _____|_____________________________________________
    |                                                 |
    v                                                 v
[STAGE 4A] qwen3-vl:4b                    [STAGE 4B] qwen3:8b
Vision Verifier (sees image)              Evidence Reviewer (sees statistics)
Checks: shape, color, markings,           Reviews: probabilities, confidence,
packaging, texture, anomalies             quality metrics -> structured JSON
    |                                                 |
    |_________________________________________________|
         |
         v
[STAGE 5] Decision Engine (4-Rule Conservative Policy)
  Threshold = 70.0% confidence
         |
         v
FINAL SCREENING RESULT
  LIKELY AUTHENTIC | LIKELY COUNTERFEIT | NEEDS MANUAL REVIEW
```

Why Dual AI Models?

Using two independent AI systems provides a safety net:

1. The CNN (EfficientNet-B0) is fast but is trained only on specific dataset distributions.
2. The Vision LLM (qwen3-vl:4b) has broad world knowledge and can detect unexpected packaging text or unusual

markings.

3. The Text LLM (qwen3:8b) provides clinical reasoning about statistical confidence and margin quality.

When both agree, we have corroboration. When they disagree, we demand manual inspection - the safest response.

3. Primary ML Model: EfficientNet-B0

What is EfficientNet-B0?

EfficientNet-B0 is a state-of-the-art Convolutional Neural Network (CNN) architecture introduced by Google Brain (Tan & Le, 2019). It uses a technique called Compound Scaling which simultaneously scales the network's depth (number of layers), width (neurons per layer), and resolution (input image size) in a principled ratio.

Key principle: instead of increasing just one dimension, compound scaling improves all three together using a fixed coefficient (phi), which gives better accuracy per parameter than scaling only depth or width.

Architecture Details

EfficientNet-B0 is built from Mobile Inverted Bottleneck Convolution (MBConv) blocks:

- Input layer: 224x224 RGB image (3 channels).
- 7 stacked MBConv blocks with Squeeze-and-Excitation (SE) attention.
- Squeeze-Excitation: channels are globally averaged, then multiplied by learned sigmoid-weighted channel attention.
- Depthwise Separable Convolution: factorizes standard 3x3 conv into depthwise (per-channel) + pointwise (1x1).
- Total Parameters: ~5.3M (base) -> our fine-tuned head: ~4.0M trainable.
- Final Pooling: Global Average Pooling (GAP) -> 1280-dimensional feature vector.
- Classifier Head: Linear(1280, 2) -> [authentic, counterfeit].

Why EfficientNet-B0 Was Chosen

- **Only 4.0M effective parameters - runs on a laptop CPU or entry-level GPU.**
- **Achieves ImageNet accuracy comparable to ResNet-50 at 8x fewer parameters.**
- **Designed for transfer learning - strong ImageNet pretrained backbone.**
- **Extremely fast inference: < 10ms per image on CPU.**
- **Proven track record in medical imaging and fine-grained visual recognition tasks.**

How Transfer Learning Works in This Project

```
STEP 1: Load EfficientNet-B0 pretrained on ImageNet-1k (1.28M images, 1000 classes)
        => Base weights: models/efficientnet_b0.pth

STEP 2: REPLACE the final classification head:
        Before: Linear(in=1280, out=1000)  -- 1000 ImageNet classes
        After:  Linear(in=1280, out=2)     -- 2 pharmaceutical classes

STEP 3: FINE-TUNE on pharmaceutical images
        - Backbone layers: updated with small learning rate
        - Classifier head: updated normally
        - Loss: Weighted CrossEntropy (to handle class imbalance)
```

```
STEP 4: SAVE best checkpoint (by validation accuracy)
        => outputs/full_data_training/checkpoints/...efficientnet_b0/best.pt
```

EfficientNet-B0 Performance Results

Metric	Value	Description
Test Accuracy	99.88%	Correct classifications on 827 held-
Test F1-Score	99.82%	Harmonic mean of precision and recal
Precision	99.65%	Of predicted counterfeits, 99.65% we
Recall (Sensitivity)	100.00%	Zero real counterfeits were missed (
Specificity	99.82%	99.82% of authentic tablets correctl
ROC-AUC	0.9988	Near-perfect discriminative power ac

Confusion Matrix on 827 test images (546 Authentic, 281 Counterfeit):

- True Positives (Counterfeit correctly flagged): 281 / 281 (100%)
- True Negatives (Authentic correctly cleared): 545 / 546 (99.8%)
- False Positives (Authentic flagged as fake): 1 / 546 (0.18%)
- False Negatives (Counterfeit missed): 0 / 281 (0.00%)

Zero missed counterfeits (0% False Negative Rate). This is the most critical safety metric - the system never lets a counterfeit through as authentic.

4. Supporting Models: MobileNetV3, Baseline CNN, GoldFormer

Why Multiple Models?

This project includes 3 additional model architectures as scientific benchmarks. The purpose is to quantitatively demonstrate WHY EfficientNet-B0 was chosen as the primary model, and to show the progression from simple to complex architectures.

Model 1: Baseline CNN (3-Layer Convolutional Network)

Architecture: Conv-BN-ReLU-MaxPool (x3) + GlobalAvgPool + Dropout(0.25) + Linear(128, 2)

- Parameters: ~94,000 (94K) - smallest model.
- Test Accuracy: 86.58% - significantly lower than transfer learning models.
- Purpose: Proves that vanilla CNNs are insufficient without pretrained knowledge.
- No ImageNet pretraining - all weights initialized randomly.

Model 2: MobileNetV3-Large

Architecture: Inverted residual blocks with Hard Swish activations, designed for mobile deployment.

- Parameters: ~4.2M - similar to EfficientNet-B0.
- Test Accuracy: 99.40% - strong, but slightly below EfficientNet-B0.
- Uses Squeeze-and-Excitation + Hard Swish instead of sigmoid activation.
- Design goal: maximum speed on mobile ARM processors.
- Latency on CPU is lower than EfficientNet-B0 (runs faster on phones).

Model 3: GoldFormer (Cross-Domain Transfer)

Architecture: Hybrid model combining ResNet50 (CNN backbone) + Swin Transformer (Swin-T) with Cross-Attention fusion. Originally designed to detect counterfeit gold coins and bullion bars.

The research hypothesis: if a model trained on detecting fake gold (different physical substrate) can learn transferable counterfeit signatures (uneven edges, discoloration, imprint irregularities), it might transfer well to pharmaceutical tablets.

- Parameters: 54,297,277 (54M) - largest model by far (207.7 MB weights).
- Test Accuracy: 99.27% - excellent, despite zero pharmaceutical pretraining.
- Validates the cross-domain transfer hypothesis - the model generalizes counterfeit detection.
- Slower to run than EfficientNet-B0 due to transformer self-attention layers.

Model	Params	Test Acc	Test F1	ROC-AUC	Notes
Baseline CNN	94K	86.58%	82.30%	0.9456	No pretrain - pure baseline
EfficientNet-B0	~4.0M	99.88%	99.82%	0.9988	PRIMARY MODEL - best accuracy
MobileNetV3-L	~4.2M	99.40%	99.11%	0.9994	Mobile-optimized alternative

GoldFormer	54M	99.27%	98.94%	0.9993	Cross-domain research benchmar

5. Dataset: Collection, Cleaning & Splits

Data Sources

The dataset was assembled from 4 independent public sources:

- NLM C3PI (National Library of Medicine, Computational Photography Project for Pill Identification): 48,312 raw authentic pharmaceutical images from NIH standardized photography.
- C3PI Reference Set: 4,392 high-quality standardized reference photographs of authentic tablet types.
- PharmAssist Dataset: 4,997 images of Indian branded medicines (authentic, diverse lighting/angles).
- Counterfeit Dataset (Roboflow/CreateML): 5,743 real-world and synthetically augmented counterfeit tablet images.

Dataset Cleaning Pipeline

Raw corpus = 68,988 images. Cleaning removed 58,437 images for these reasons:

- Near-Duplicate Removal: 51,794 images eliminated using perceptual hashing (pHash) - grouped by visual similarity, kept only 1 per group.
- Unverified Labels: 1,683 images with ambiguous or conflicting label metadata removed.
- SHA-256 Deduplication: Exact byte-level duplicates removed between train/validation/test splits to prevent data leakage.

Final Dataset Counts

Split	Images	Authentic	Counterfeit	Usage
Training	3,858	2,519	1,339	Model weight updates (backpropagati
Validation	827	541	286	Checkpoint selection (early stoppin
Test (Held-Ou	827	546	281	Final one-time evaluation (never se
Self-Supervis	5,039	-	-	Used for Baseline CNN pretraining o
TOTAL	10,551	3,602	1,910	Full pipeline image pool

Data Leakage Prevention

Data leakage occurs when training data 'leaks' into the test set, causing artificially inflated accuracy. This project uses two layers of leakage prevention:

1. SHA-256 exact hash checking: any image whose exact content appears in train.csv is excluded from test.csv.
2. Perceptual hash grouping: near-visually-identical images are assigned to only ONE of train/val/test - never split across splits.

A clean test evaluation report (reports/clean_test_evaluation.md) documents the 5 exact images that were removed from the test set because their SHA-256 hash matched training images.

6. Training Process & Hyperparameters

Training Configuration

Hyperparameter	Value	Explanation
Optimizer	AdamW	Adaptive Moment Estimation + Weight Decay r
Learning Rate	3e-4 (0.0003)	Base rate; Adam adapts per-parameter
Weight Decay	1e-2 (0.01)	L2 regularization to prevent overfitting
Batch Size	8 physical x 2 acc	Effective batch = 16 images per gradient st
Loss Function	Weighted CrossEntr	Weights compensate 1.88:1 authentic:counter
Max Epochs	20	Upper limit with early stopping (patience=4
EfficientNet Epochs	5 (actual)	Best validation checkpoint saved at epoch 5
Input Size	224 x 224 RGB	ImageNet standard resolution
Device	CUDA (RTX 3050)	NVIDIA GPU training, CPU fallback available

Data Augmentation (Training Only)

Augmentation applies random transforms at training time to improve generalization. Never applied during evaluation.

- Random Horizontal Flip (p=0.5): mirrors tablet left-right to simulate different orientations.
- Random Affine Rotation (+/-8 degrees): accounts for slightly tilted photographs.
- Color Jitter: Brightness +/-15%, Contrast +/-15%, Saturation +/-10% - simulates different lighting conditions.
- No augmentation is applied at validation/test time (pure evaluation).

Class Imbalance Handling

The authentic class has 1.88x more images than counterfeit. Without correction, the model could learn to always predict authentic and still achieve ~65% accuracy. Fix: weighted CrossEntropyLoss where counterfeit images are assigned higher loss weight:

weight_authentic = 1 / (freq_authentic) = 1 / 0.653 = 1.53

weight_counterfeit = 1 / (freq_counterfeit) = 1 / 0.347 = 2.88

This forces the model to penalize missed counterfeits more heavily.

Self-Supervised Pretraining (Baseline CNN Only)

Before supervised training, the Baseline CNN is pretrained on 5,039 UNLABELED images using SimCLR-style contrastive learning:

- Two random augmented views of the same image are created.
- The network learns to produce similar embeddings for the two views of the same image.
- This teaches useful visual representations without any labels.
- The pretrained backbone is then fine-tuned with labeled pharmaceutical images.

EfficientNet-B0 and MobileNetV3 do NOT need self-supervised pretraining because they already have ImageNet pretrained weights which provide much stronger initialization.

7. Model Evaluation Results

Key Terms Explained

- Accuracy: (TP + TN) / Total. Percentage of all predictions that were correct.
- Precision: TP / (TP + FP). Of all images flagged as counterfeit, how many truly were counterfeit?
- Recall (Sensitivity): TP / (TP + FN). Of all real counterfeits, how many did we catch? (MOST IMPORTANT for safety)
- Specificity: TN / (TN + FP). Of all authentic tablets, how many were correctly cleared?
- F1-Score: 2 x (Precision x Recall) / (Precision + Recall). Harmonic mean - balances both precision and recall.
- ROC-AUC: Area Under the Receiver Operating Characteristic Curve. 1.0 = perfect classifier.
- False Negative: A real counterfeit classified as authentic. MOST DANGEROUS ERROR in pharmaceutical safety.
- False Positive: A real authentic tablet flagged as counterfeit. Less dangerous but causes unnecessary concern.

EfficientNet-B0 Confusion Matrix (Primary Model, 827 test images)

```
                    PREDICTED:          PREDICTED:
                    Authentic           Counterfeit
ACTUAL: Authentic       545 (TN)            1 (FP)   <- 1 authentic wrongly flagged
ACTUAL: Counterfeit       0 (FN)          281 (TP)   <- ALL 281 counterfeits caught!

False Negative Rate = 0 / 281 = 0.00%  <-- ZERO missed counterfeits
False Positive Rate = 1 / 546 = 0.18%  <-- Only 1 false alarm
```

Confidence Threshold Policy

The system uses a confidence threshold of 70% (T = 0.70):

- If model confidence >= 70% AND model says AUTHENTIC -> Status: AUTHENTIC
- If model confidence >= 70% AND model says COUNTERFEIT -> Status: COUNTERFEIT
- If model confidence < 70% (regardless of prediction) -> Status: NEEDS MANUAL REVIEW

This means low-certainty predictions are automatically escalated to manual inspection, preventing the system from making uncertain claims.

8. Secondary AI: qwen3-vl:4b Vision Verifier

What is qwen3-vl:4b?

qwen3-vl:4b is a multimodal Vision-Language Model developed by Alibaba Cloud (Qwen Team). It can simultaneously process both images and text, making it capable of answering visual questions about the content of an image.

Model specifications:

- Parameters: 4.4 Billion (4B) - quantized to 4-bit (Q4_K_M) = 3.3 GB disk size.
- Capabilities: Vision (image input), Text generation, Tool use, Internal reasoning (Thinking mode).
- Context: Up to 262,144 tokens context window.
- Architecture: Qwen3 decoder-only transformer with a visual encoder module.
- Runs locally via Ollama - no internet required. Runs entirely on your machine (RTX 3050 + 16GB RAM).

How the Vision Verifier Works

- The input image is loaded and resized to maximum 384x384 pixels (to fit in 4GB VRAM).
- The image is encoded to base64 string format.
- The base64 image + a structured inspection prompt are sent to Ollama (http://localhost:11434/api/generate).
- qwen3-vl:4b reads the image directly and generates a visual inspection report.
- The response is parsed to extract: Assessment, Visual Confidence, Image Quality, Observations, Concerns.

What the Vision Verifier Examines

- Tablet Shape: Is it round, oval, oblong? Does it look symmetrical and uniform?
- Tablet Color: Is the color uniform? Any discoloration, unusual spots, or coating issues?
- Surface Texture: Is the surface smooth/glossy? Any bubbles, cracks, pitting, or rough patches?
- Imprints/Markings: Are letter imprints sharp and clear? Distorted imprints can indicate counterfeit dies.
- Edges: Are edges uniform? Chipping or uneven edges suggest poor manufacturing quality.
- Packaging (if visible): Is label text correctly printed? Any spelling errors, low-quality printing?

Hardware Constraints & Timing

On the RTX 3050 Laptop (4 GB VRAM) + 16 GB system RAM:
- Inference time per image: approximately 65-80 seconds
- Timeout limit: 180 seconds (3 minutes)
- Token budget: 750 output tokens (prevents timeout while allowing full report)
- Image resize limit: 384 px max (reduces vision encoder token count)

Why so slow? Vision-Language models process images by converting them to visual tokens (patches), which are expensive to compute on a small 4GB VRAM laptop GPU. Ollama offloads layers between VRAM and system RAM, which adds memory bandwidth latency.

When Vision Verifier Shows UNAVAILABLE

- **Ollama service is offline (connection refused at http://localhost:11434).**
- **Inference timed out after 180 seconds (very large image or GPU overloaded).**
- **Model not installed (e.g. qwen3-vl:8b requested but only :4b is present).**

In all UNAVAILABLE cases, the system falls back safely: the Decision Engine uses only the primary EfficientNet-B0 result and attaches an explicit warning to the output.

9. Secondary AI: qwen3:8b Evidence Reviewer

What is qwen3:8b?

qwen3:8b is a text-only Large Language Model (LLM) from Alibaba Cloud. It does NOT see the image. Instead, it receives the structured statistical output from the EfficientNet-B0 classifier as a JSON evidence payload and acts as an analytical second opinion.

- Parameters: 8.2 Billion (8B) - 5.2 GB at Q4_K_M quantization.
- Capabilities: Advanced text reasoning, chain-of-thought, structured output, tool use.
- Context: 40,960 tokens.
- Role: Clinical-style reasoning reviewer over probability statistics.
- Location: Runs locally via Ollama (http://localhost:11434). TEXT-ONLY - no vision capability.

Evidence Payload Sent to qwen3:8b


```
{
  "model": "efficientnet_b0",
  "prediction": "counterfeit",
  "authentic_probability": 0.0019,
  "counterfeit_probability": 0.9981,
  "confidence": 0.9981,
  "threshold": 0.70,
  "image_quality": "good",
  "quality_checks": {
    "resolution_ok": true,
    "blur_ok": true,
    "brightness_ok": true
  }
}
```


qwen3:8b Structured Response


```
{
  "assessment": "COUNTERFEIT",
  "confidence": 0.95,
  "reasoning": [
    "Counterfeit probability 99.81% is extremely high.",
    "Confidence exceeds 70% threshold by 29.81 points.",
    "No quality issues detected - classification is reliable."
  ],
  "needs_human_verification": true,
  "warning": "Visual analysis cannot establish chemical authenticity."
}
```


Role in the FastAPI Backend

In the FastAPI backend (/predict and /verify endpoints), qwen3:8b is the secondary reviewer, not the vision verifier. The backend uses qwen3:8b to produce explanatory reasoning in the API response. The CLI test.py uses qwen3-vl:4b for direct visual verification. Both serve as safety layers over the primary EfficientNet-B0 classifier.

10. Decision Engine: 4-Rule Conservative Policy

Purpose

The Decision Engine is the final arbiter of all screening results. It implements a strictly conservative policy: when in doubt, always escalate to manual inspection. It combines outputs from the primary ML classifier and both secondary AI systems into a single, justified final verdict.

File: ml/verification/decision_engine.py

The 4 Decision Rules

Rule	Trigger	Final Status	Human Needed
1	Image quality failed (blur/re	NEEDS MANUAL REVIEW	YES
2	Primary confidence < 70%	NEEDS MANUAL REVIEW	YES
3A	High conf + Both say AUTHENTI	LIKELY AUTHENTIC	NO
3B	High conf + Both say COUNTERF	LIKELY COUNTERFEIT	YES
3C	High conf + Qwen UNCERTAIN	NEEDS MANUAL REVIEW	YES
3D	High conf + Models DISAGREE	NEEDS MANUAL REVIEW	YES
4A	High conf + AI offline + Auth	LIKELY AUTHENTIC (warned	NO
4B	High conf + AI offline + Coun	LIKELY COUNTERFEIT	YES

Why Conservative Design?

A conservative policy means that whenever there is any uncertainty, the system errs on the side of safety (demanding manual review) rather than giving a confident but potentially wrong verdict.

In pharmaceutical safety, a missed counterfeit (False Negative) is far more dangerous than a false alarm (False Positive). The Decision Engine encodes this asymmetry by requiring high confidence AND corroboration from the secondary AI before clearing a medicine as LIKELY AUTHENTIC.

11. FastAPI Backend & API Endpoints

What is FastAPI?

FastAPI is a modern, high-performance web framework for building REST APIs in Python. It uses async I/O for handling many concurrent requests, and automatically generates interactive API documentation (Swagger UI) at /docs.

The backend server is the production interface for the screening system, allowing mobile apps, web frontends, or laboratory information systems to submit images and receive results.

Starting the Backend Server

```
# Run from project root directory:
.venv\Scripts\uvicorn.exe backend.app.main:app --host 127.0.0.1 --port 8000

# Access Swagger UI at:
http://127.0.0.1:8000/docs

# API Health Check:
http://127.0.0.1:8000/health
```

API Endpoints

GET /health (or GET /)

Returns server status, device (CPU/CUDA), loaded models, and Ollama configuration.

```
{
  "status": "ok", "version": "1.0.0",
  "device": "cuda", "default_model": "efficientnet_b0",
  "available_models": ["efficientnet_b0","mobilenet_v3_large","baseline_cnn","goldformer"],
  "secondary_ai_model": "qwen3:8b",
  "secondary_ai_endpoint": "http://localhost:11434"
}
```

POST /predict or POST /verify

Upload a tablet image and receive full dual-model screening result.

Form parameters:

- file: The tablet image (JPEG, PNG, WEBP, or BMP format).
- threshold: Confidence threshold (default: 0.70).
- model: Model name to use (default: efficientnet_b0).
- enable_qwen: true/false - whether to run secondary AI review (default: true).

Response includes: prediction, confidence, qwen assessment, final_result, needs_human_verification, disclaimer.

Backend File Structure

- **backend/app/main.py - FastAPI app, lifespan (model preloading), CORS, health endpoint.**

- backend/app/routes/predict.py - POST /predict and POST /verify route handlers.
- backend/app/services/classifier.py - ClassifierService singleton: loads model, calls qwen reviewer.
- backend/app/schemas/predict.py - Pydantic schemas (PredictionResponse, HealthResponse) with strict typing.

12. Image Quality Preprocessing & Checks

Quality Precheck (Before CNN Inference)

Every image is first passed through an OpenCV-based quality filter (ml/preprocessing/quality_checker.py):

Check	Method	Threshold	Failure -> Action
Minimum Resolut	img.shape[0], img.shape[1]	>= 64 x 64	NEEDS MANUAL REVIEW
Blur Detection	cv2.Laplacian(gray, CV_64F)	>= 50.0	NEEDS MANUAL REVIEW
Brightness	np.mean(gray_image)	15.0 to 245.0	NEEDS MANUAL REVIEW

Laplacian Variance for blur detection: The Laplacian operator computes the second derivative of image intensity. For sharp images, edges produce high second derivatives (high variance). Blurry images have smooth, gradual transitions - low Laplacian variance.
Threshold: variance < 50 = too blurry to reliably classify.

Image Preprocessing for CNN Input

After quality checks pass, the image is transformed for model input (ml/preprocessing/image_preprocessor.py):

- Read image with PIL.Image.open() -> convert to RGB (3 channels).
- Resize to exactly 224 x 224 pixels using BILINEAR interpolation.
- Convert to float32 tensor, scale pixel values from [0, 255] to [0.0, 1.0].
- Normalize: subtract ImageNet mean [0.485, 0.456, 0.406], divide by std [0.229, 0.224, 0.225].
- Add batch dimension: shape changes from (3, 224, 224) to (1, 3, 224, 224).
- Send to GPU/CPU and pass through model.forward().

Why ImageNet Normalization?

EfficientNet-B0 was pretrained on ImageNet-1k where all 1.28 million training images were normalized with these exact mean and std values. When we fine-tune on pharmaceutical images, we must apply the SAME normalization so the pretrained convolutional weights receive inputs in the same numerical range they were trained on. Without this, the pretrained features would produce meaningless activations.

13. Testing & Automated Test Suite

Running Tests

```
# Run all 102 tests:
.venv\Scripts\pytest -v

# Run specific test file:
.venv\Scripts\pytest tests/test_api.py -v

# Result (verified):
102 passed, 10 warnings in 63.04s
```

Test Suite Breakdown

Test File	Tests	What is Verified
test_api.py	9	FastAPI /health, /predict, /verify, error codes (400
test_decision_engine.py	16	All 4 Decision Engine rules, conflict resolution, fa
test_qwen_verifier.py	21	JSON parsing, markdown stripping, timeouts, 404 erro
test_qwen_vision_verifier.p	10	Vision verifier parsing, synthesize_final_result, fo
test_unified_models.py	8	Forward-pass shapes for all 4 architectures, model l
test_transfer_learning.py	8	Classifier shapes, training metadata contract, Image
test_quality.py	6	Resolution, blur, brightness, missing file, extreme
test_features.py	3	Handcrafted feature extraction, determinism, feature
test_preprocessing.py	4	Image pipeline, format support, invalid paths, deter
test_evaluate.py	1	Test manifest loading and integrity verification
TOTAL	102	100% passing

14. Project File Structure

```
CounterfeitPharmaceuticalDetection/
|
+-- backend/                    <- FastAPI REST API
|   +-- app/
|       +-- main.py             <- FastAPI app, startup, health endpoint
|       +-- routes/predict.py   <- POST /predict, POST /verify
|       +-- services/classifier.py <- ClassifierService singleton
|       +-- schemas/predict.py  <- Pydantic response schemas
|
+-- ml/                         <- Machine Learning pipeline
|   +-- config.py               <- Model registry, paths, thresholds
|   +-- models/transfer_learning.py <- Build & load all 4 architectures
|   +-- inference/predict.py    <- Single image & batch prediction CLI
|   +-- preprocessing/
|   |   +-- image_preprocessor.py  <- Resize, normalize, tensor convert
|   |   +-- quality_checker.py     <- Blur/resolution/brightness checks
|   +-- training/
|   |   +-- train.py            <- Main supervised training loop
|   |   +-- data.py             <- Dataset class, transforms, augmentation
|   +-- evaluation/
|   |   +-- evaluate.py         <- Test set evaluation, metrics
|   |   +-- benchmark_models.py <- Compare all 4 models side-by-side
|   +-- verification/
|       +-- qwen_verifier.py    <- qwen3-vl:4b Vision Verifier (CLI)
|       +-- qwen_reviewer.py    <- qwen3:8b Evidence Reviewer (API)
|       +-- decision_engine.py  <- 4-rule conservative Decision Engine
|
+-- models/                     <- Base pretrained weights
|   +-- efficientnet_b0.pth     <- ImageNet pretrained EfficientNet-B0
|   +-- mobilenet_v3_large.pth  <- ImageNet pretrained MobileNetV3-Large
|
+-- test_images/                <- Reference test images for demonstration
+-- tests/                      <- pytest automated test suite (102 tests)
+-- data/                       <- Dataset manifests and split CSVs
+-- outputs/                    <- Training checkpoints and metrics
+-- external_models/            <- GoldFormer architecture and weights
+-- test.py                     <- Main CLI: test single image or directory
+-- FINAL_PROJECT_AUDIT_REPORT.md <- Full technical audit report
+-- requirements.txt            <- All Python dependencies
+-- .env.example                <- Environment configuration template
```

15. Live Demo: Running the System

Prerequisites

- **Virtual environment activated: run .venv\Scripts\activate**
- **Ollama running locally: check with curl http://localhost:11434**
- **qwen3-vl:4b installed in Ollama: run ollama list**

Test a Single Image (CLI)

```
python test.py test_images\verified_counterfeit.jpg

Expected output:
==================================================
PHARMACEUTICAL ML CLASSIFIER
============================
Image: verified_counterfeit.jpg
Prediction: COUNTERFEIT
Authentic probability: 0.19%
Counterfeit probability: 99.81%
Confidence: 99.81%
Status: COUNTERFEIT

==================================================
QWEN VISION VERIFICATION
========================
Assessment: COUNTERFEIT
Visual Confidence: HIGH
Image Quality: GOOD
Tablet Analysis:
* Shape: Irregular
* Color: Non-uniform
...
Recommendation: MANUAL REVIEW

==================================================
FINAL SCREENING RESULT
======================
Primary ML Prediction: COUNTERFEIT
Qwen Visual Assessment: COUNTERFEIT
Final Status: LIKELY COUNTERFEIT
```

Test All Images in Batch (CLI, Fast Mode)

```
python test.py test_images --no-vision
```

```
# Runs primary EfficientNet-B0 on all 11 images in test_images/ folder
# Saves results to outputs/predictions.csv
```

Start FastAPI Server & Use Swagger UI

```
.venv\Scripts\uvicorn.exe backend.app.main:app --host 127.0.0.1 --port 8000

# Open in browser: http://127.0.0.1:8000/docs
# Use the /predict POST endpoint to upload any tablet image
# Health check: http://127.0.0.1:8000/health
```

16. Limitations & Safety Boundary

Critical Limitations to Know

- Visual-Only: Cannot detect wrong active pharmaceutical ingredients (APIs), under-dosing, or harmful substitutes. A perfectly manufactured counterfeit (identical appearance) would NOT be caught.

- Photography Conditions: Specular highlights from foil packaging, blurry images, or extreme lighting degrade CNN performance.

- Domain Shift: Training images are predominantly high-quality clinical captures (C3PI). In-field smartphone images may have different quality characteristics.

- Confidence != Chemical Purity: A 99% confidence score means the tablet LOOKS like authentic tablets in the training set - not that its chemical composition is certified.

- No Barcode / NDC Validation: The system does not read or verify National Drug Codes, GS1 barcodes, or serialization numbers.

- Ollama Dependency: The secondary AI requires a locally running Ollama instance. In offline deployment without Ollama, only the primary CNN result is available.

- GPU Memory: Running qwen3-vl:4b requires offloading between 4GB VRAM and 16GB RAM. On machines with less than 16GB RAM, inference may fail.

LEGAL & REGULATORY NOTICE: This system is an AI-assisted research screening tool for academic study. It is NOT approved for clinical, regulatory, or commercial pharmaceutical quality control. Laboratory chemical analysis (HPLC, mass spectrometry, NIR spectroscopy) remains the only legally accepted method for pharmaceutical authentication.

17. Future Improvements

- **OCR Integration: Add PaddleOCR or Tesseract to extract NDC numbers and batch codes from packaging, enabling cross-referencing against national drug databases.**
- **Barcode Scanning: Integrate GS1 DataMatrix 2D barcode scanning and validation against the Drug Supply Chain Security Act (DSCSA) serialization database.**
- **Mobile Deployment: Quantize EfficientNet-B0 to 8-bit integers (INT8 via ONNX Runtime / TensorFlow Lite) for on-device inference on Android/iOS without cloud connectivity.**
- **NIR Spectroscopy: Pair smartphone camera triage with portable near-infrared (NIR) spectroscopy sensors for combined visual + molecular verification.**
- **Multi-Drug Training: Expand training dataset to include more Indian generic brands and regional pharmaceutical packaging styles.**
- **Active Learning: Deploy with human-in-the-loop feedback loop - pharmacists validate borderline NEEDS MANUAL REVIEW cases and those become new training data.**
- **Federated Learning: Train across multiple pharmacy chains without sharing raw patient/drug images (privacy-preserving).**
- **Explainability (Grad-CAM): Generate heatmaps showing WHICH regions of the tablet image influenced the classification decision.**

18. Potential Exam Questions & Answers

Q: What is the main objective of this project?

A: To build an AI-assisted smartphone screening system that uses computer vision to detect potentially counterfeit pharmaceutical tablets, providing a preliminary safety verdict before expensive laboratory testing.

Q: What is EfficientNet-B0 and why was it chosen?

A: EfficientNet-B0 is a compound-scaled CNN from Google Brain (2019). It was chosen because it achieves near state-of-the-art accuracy with only ~4M parameters, making it fast to run on limited hardware. It achieved 99.88% test accuracy on our pharmaceutical dataset.

Q: What is transfer learning and how is it used here?

A: Transfer learning reuses a model pretrained on a large dataset (ImageNet, 1.28M images, 1000 classes) and fine-tunes it on a smaller task-specific dataset. Here, we replace EfficientNet-B0's final Linear(1280, 1000) head with Linear(1280, 2) and train on 3,858 pharmaceutical images.

Q: What does 99.88% test accuracy mean? Does it guarantee authenticity?

A: It means 99.88% of 827 held-out test images were correctly classified. It does NOT guarantee chemical authenticity - it only means the tablet LOOKS visually like authentic tablets in the training distribution. A chemically adulterated tablet that appears identical would not be detected.

Q: What is the role of qwen3-vl:4b?

A: qwen3-vl:4b is a multimodal Vision-Language Model that directly analyzes the tablet image. It examines physical properties like shape, color uniformity, imprint clarity, surface texture, and packaging to provide a secondary visual opinion independent of the CNN.

Q: What is the role of qwen3:8b?

A: qwen3:8b is a text-only LLM that receives structured statistical evidence (probabilities, confidence score, quality metrics) from the primary CNN as a JSON payload and provides a reasoned analytical review. It does NOT see the image directly.

Q: What is the Decision Engine and why is it conservative?

A: The Decision Engine combines all model outputs using 4 rules. It is conservative because it always demands manual review when there is uncertainty, model disagreement, or low confidence. This prevents false certainty - a missed counterfeit is more dangerous than a false alarm.

Q: What is data leakage and how is it prevented?

A: Data leakage occurs when training images accidentally appear in the test set, inflating accuracy metrics. We prevent it with SHA-256 exact hash checking (byte-level) and perceptual hash grouping (visually similar images go to only one split).

Q: What happens if Ollama is offline?

A: The system falls back gracefully. It continues with only the EfficientNet-B0 primary result and attaches a warning 'Secondary AI verification unavailable - result based on primary classifier only.' It never crashes.

Q: What is the confidence threshold and why 70%?

A: The threshold T=70% means predictions below 70% confidence are automatically escalated to NEEDS MANUAL REVIEW, regardless of predicted class. 70% was chosen as a balance - high enough to avoid uncertain verdicts but not so high that most real images get rejected.

Q: Why does the system have multiple models (EfficientNet, MobileNet, Baseline CNN, GoldFormer)?

A: Multiple models serve as scientific benchmarks. Baseline CNN (86.58%) shows that simple CNNs are insufficient without transfer learning. MobileNetV3 (99.40%) and GoldFormer (99.27%) demonstrate that multiple architectures achieve near-identical results, validating that EfficientNet-B0's 99.88% is genuine.

Q: What is Compound Scaling in EfficientNet?

A: Compound scaling simultaneously scales CNN depth (layers), width (channels), and input resolution by a fixed coefficient phi. This gives better accuracy-per-parameter than scaling only one dimension. EfficientNet-B0 is the base (phi=1); B1-B7 are progressively larger scaled versions.

Q: What is Squeeze-and-Excitation (SE) in MBConv blocks?

A: SE is a channel attention mechanism. After depthwise convolution, channels are globally average-pooled (Squeeze), then passed through two fully connected layers with sigmoid (Excitation) to produce per-channel weights. Channels with more relevant features are amplified; less relevant channels are suppressed.

Q: What dataset was used and what are the class proportions?

A: 5,512 supervised images: 3,602 authentic (65.3%) and 1,910 counterfeit (34.7%). Sources: NLM C3PI, PharmAssist Indian medicines dataset, and Roboflow/CreateML counterfeit datasets. Additional 5,039 images used for self-supervised pretraining.

Q: How is class imbalance handled?

A: Weighted CrossEntropyLoss is used. Counterfeit images receive higher loss weight (1/0.347 = 2.88) while authentic images receive lower weight (1/0.653 = 1.53). This forces the model to penalize missed counterfeits more heavily during training.

Q: What is the FastAPI backend used for?

A: FastAPI provides a REST API interface for the system. POST /predict accepts multipart image uploads and returns a full dual-model screening result as JSON. This allows mobile apps, web applications, or hospital information systems to query the screening system programmatically.

Q: What is Recall and why is it the most important metric here?

A: Recall = TP / (TP + FN). It measures what fraction of actual counterfeits were detected. A False Negative (missed counterfeit) is catastrophic - a patient would consume a dangerous counterfeit medicine thinking it was authenticated. Our system

achieves 100% Recall (0 missed counterfeits).

Q: What are the limitations of this system?

A: 1) Visual-only - cannot detect chemical composition changes. 2) Perfectly manufactured counterfeits (identical appearance) would not be detected. 3) Performance degrades with poor lighting, blur, or unusual photography angles. 4) Not clinically approved - for academic demonstration only.